

SIMATS ENGINEERING

ASSESSMENT TOOL 1: INDUSTRY PROBLEM SOLVING TASKS

COURSE NAME:	Programming in Java	COURSE CODE:	CSA0915- SLOT B
NAME:	S.K.Deepa Arathi	REG NO:	192571020

Assessment Tasks

1. Industry Scenario:

A hospital wants to develop a desktop-based appointment management system for its reception counter.

Question:

Design and develop a Java AWT application that allows the receptionist to register a patient appointment. The application should provide fields for Patient ID, Patient Name, Age, Gender, Department, Doctor Name, and Appointment Date. Use appropriate AWT components such as Label, TextField, CheckboxGroup, Choice, Button, and Frame.

When the Book Appointment button is clicked, validate the entered information and generate an appointment summary containing the patient details, selected department, doctor, and appointment date. Provide a Clear button to reset all fields.

Requirements:

1. Use suitable AWT components and a container.
2. Use an appropriate layout.
3. Handle button events.
4. Validate mandatory fields.
5. Display the appointment confirmation.
6. Provide Clear functionality.

Test Cases:

tc1_input- P101, Ravi, 35, Male, Cardiology, Dr. Kumar
tc1_output- Appointment Booked Successfully tc2_input—
P102, Priya, 28, Female, Neurology, Dr. Meena tc2_output-
Appointment Booked Successfully tc3_input- Patient Name
empty tc3_output- Patient Name Required tc4_input-
Patient ID empty tc4_output- Patient ID Required
tc5_input-Age = 0
tc5_output-Invalid Age

CODE:

```
import java.awt.*; import
java.awt.event.*;
public class HospitalAppointment extends Frame implements ActionListener
{
    Label patientIdLabel, nameLabel, ageLabel, genderLabel;
    Label departmentLabel, doctorLabel, dateLabel, summaryLabel;
    TextField patientIdField, nameField, ageField, dateField;
    Checkbox male, female, other;
    CheckboxGroup genderGroup;

    Choice departmentChoice, doctorChoice;

    Button bookButton, clearButton;

    TextArea summaryArea;
    public HospitalAppointment()
    {
        setTitle("Hospital Appointment Management
System");        setSize(650, 550);        setLayout(new
BorderLayout(10, 10));

        Label title = new Label("HOSPITAL APPOINTMENT MANAGEMENT SYSTEM",
Label.CENTER);        title.setFont(new Font("Arial", Font.BOLD, 20));
add(title, BorderLayout.NORTH);

        Panel formPanel = new Panel();
formPanel.setLayout(new GridLayout(7, 2, 10, 10));

        patientIdLabel = new Label("Patient ID:");
patientIdField = new TextField();

        formPanel.add(patientIdLabel);
formPanel.add(patientIdField);

        nameLabel = new Label("Patient
Name:");        nameField = new TextField();
```

```

formPanel.add(nameLabel);
formPanel.add(nameField);

        ageLabel = new Label("Age:");
ageField = new TextField();

formPanel.add(ageLabel);
formPanel.add(ageField);
                genderLabel = new
Label("Gender:");

        Panel genderPanel = new Panel(new FlowLayout(FlowLayout.LEFT));
        genderGroup = new
CheckboxGroup();
        male = new Checkbox("Male", genderGroup, false);
female = new Checkbox("Female", genderGroup, false);
other = new Checkbox("Other", genderGroup, false);
        genderPanel.add(male);
genderPanel.add(female);
genderPanel.add(other);

formPanel.add(genderLabel);
formPanel.add(genderPanel);

        departmentLabel = new Label("Department:");
departmentChoice = new Choice();
        departmentChoice.add("Select
Department");
departmentChoice.add("Cardiology");
departmentChoice.add("Neurology");
departmentChoice.add("Orthopedics");
departmentChoice.add("Pediatrics");
departmentChoice.add("Dermatology");
        formPanel.add(departmentLabel);
formPanel.add(departmentChoice);

                doctorLabel = new Label("Doctor
Name:");

```



```

        doctorChoice = new Choice();
        doctorChoice.add("Select
Doctor");
        doctorChoice.add("Dr.
Kumar");
        doctorChoice.add("Dr.
Meena");
        doctorChoice.add("Dr.
Raj");
        doctorChoice.add("Dr.
Priya");

        formPanel.add(doctorLabel);
formPanel.add(doctorChoice);

        dateLabel = new Label("Appointment
Date:");
        dateField = new TextField();

formPanel.add(dateLabel);
formPanel.add(dateField);
        add(formPanel,
BorderLayout.CENTER);

        Panel bottomPanel = new Panel(new BorderLayout(5, 5));
        Panel buttonPanel = new Panel();
        bookButton = new Button("Book
Appointment");
        clearButton = new
Button("Clear");
        bookButton.addActionListener(this);
clearButton.addActionListener(this);
        buttonPanel.add(bookButton);
buttonPanel.add(clearButton);

        bottomPanel.add(buttonPanel, BorderLayout.NORTH);

        summaryLabel = new Label("Appointment Summary:");
bottomPanel.add(summaryLabel, BorderLayout.CENTER);

        summaryArea = new TextArea("", 8, 60,
                                TextArea.SCROLLBARS_VERTICAL_ONLY);
summaryArea.setEditable(false);
        bottomPanel.add(summaryArea,
BorderLayout.SOUTH);

```



```

        add(bottomPanel, BorderLayout.SOUTH);
        addWindowListener(new WindowAdapter()
{
    public void windowClosing(WindowEvent e)
{
        dispose();
    }
});

setVisible(true);
    }    public void
actionPerformed(ActionEvent e) {

    if (e.getSource() == bookButton) {
bookAppointment();
    }
    if (e.getSource() == clearButton)
{
        clearFields();
    }
}

private void bookAppointment() {

    String patientId = patientIdField.getText().trim();
    String patientName = nameField.getText().trim();
    String ageText = ageField.getText().trim();
    String date = dateField.getText().trim();
        if (patientId.isEmpty()) {
summaryArea.setText("Patient ID Required");
return;
        }

    if (patientName.isEmpty()) {
        summaryArea.setText("Patient Name Required");
return;
    }

    if
(ageText.isEmpty()) {

```



```

        summaryArea.setText("Age Required");
return;
    }
    int
age;
    try {
        age =
Integer.parseInt(ageText);
        if (age <= 0 || age > 120) {
summaryArea.setText("Invalid Age");
return;
        }

    } catch (NumberFormatException ex) {
summaryArea.setText("Invalid Age");
return;
    }

    Checkbox selectedGender = genderGroup.getSelectedCheckbox();
    if (selectedGender == null) {
summaryArea.setText("Gender Required");
return;
    }

    if (departmentChoice.getSelectedIndex() ==
0) {
        summaryArea.setText("Department
Required");
return;
    }

    if (doctorChoice.getSelectedIndex() == 0) {
summaryArea.setText("Doctor Name Required");
return;
    }

    if (date.isEmpty()) {
summaryArea.setText("Appointment Date Required");
return;
    }

```

```

String gender = selectedGender.getLabel();
String department = departmentChoice.getSelectedItemAt();
String doctor = doctorChoice.getSelectedItemAt();

summaryArea.setText(
    "Appointment Booked Successfully\n\n" +
    "Patient ID      : " + patientId + "\n" +
    "Patient Name    : " + patientName + "\n" +
    "Age             : " + age + "\n" +
    "Gender          : " + gender + "\n" +
    "Department      : " + department + "\n" +
    "Doctor Name     : " + doctor + "\n" +
    "Appointment Date : " + date
);
}

private void
clearFields() {

patientIdField.setText("");
nameField.setText("");
ageField.setText("");
dateField.setText("");

genderGroup.setSelectedCheckbox(null);

departmentChoice.select(0);
doctorChoice.select(0);

summaryArea.setText("");
}

    public static void main(String[] args)
    {
        new
HospitalAppointment();
    }
}

```